

Android Development: LinearLayout Masterclass

Complete Study Guide & Reference Notes | Created by AI Tutor

1. What is LinearLayout?

LinearLayout is a fundamental View Container in Android XML that aligns all of its children in a single direction — either **Vertically** (top-to-bottom) or **Horizontally** (left-to-right).

- **android:orientation="vertical"**: Stacks child views on top of each other.
- **android:orientation="horizontal"**: Places child views side-by-side from left to right.

2. Gravity vs. Layout_Gravity

Understanding alignment is essential for building clean layouts:

- **android:gravity**: Controls where the **children or internal content** go inside the container (e.g. centering text inside a TextView, or centering all views inside a LinearLayout).
- **android:layout_gravity**: Controls where **this specific view** positions itself relative to its parent container.

3. Proportion Control: layout_weight & weightSum

Weights allow child views to divide available space proportionally like slices of a pie:

- **Rules for Weight**: When using `android:layout_weight`, set `android:layout_width="0dp"` for horizontal layouts, or `android:layout_height="0dp"` for vertical layouts.
- **Equal Split**: Giving two horizontal children `layout_weight="1"` splits the screen exactly 50% / 50%.
- **Proportional Split**: Weights of 1 and 2 result in a 33.3% and 66.7% split (total weight = 3).
- **android:weightSum**: Defines the explicit total pool of points available on the parent layout (e.g. setting `weightSum="2"` and `weight="1"` on a single child forces it to occupy exactly 50% of the screen width).

4. Intermediate & Performance Features

A. Vertical Weights for Dynamic Spacing

By setting a middle container's height to 0dp and `layout_weight="1"`, while keeping the top header and bottom footer as `wrap_content`, the middle section dynamically expands to fill all remaining vertical space on any phone screen.

B. Built-in Dividers

Instead of manually creating divider lines, LinearLayout provides built-in system dividers:

```
android:showDividers="middle"
```

```
android:divider="?android:attr/listDivider"
```

The `?android:attr/` syntax tells Android to dynamically fetch the default line color defined by the user's system theme (light/dark mode).

C. Performance Optimization: baselineAligned

Horizontal LinearLayouts attempt to align the text baselines of all child views by default. Setting `android:baselineAligned="false"` on horizontal layouts with complex children disables this calculation, saving battery and rendering time.

5. Masterclass Layout Code (Scoreboard Example)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="10dp"
    android:showDividers="middle"
    android:divider="?android:attr/listDivider">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="MATCH SCORE"
        android:textSize="24sp"
        android:layout_gravity="center" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:gravity="center"
        android:baselineAligned="false"
        android:orientation="horizontal"
        android:showDividers="middle"
        android:divider="?android:attr/listDivider"
        android:weightSum="2">

        <!-- Team A Column -->
        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:gravity="center">
            <TextView android:text="TEAM A" android:textSize="22sp" />
            <TextView android:id="@+id/scoreDisplayA" android:text="0" android:textSize="36sp" />
            <Button android:id="@+id/incTeamA" android:text="+1" />
        </LinearLayout>

        <!-- Team B Column -->
        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:gravity="center">
            <TextView android:text="TEAM B" android:textSize="22sp" />
            <TextView android:id="@+id/scoreDisplayB" android:text="0" android:textSize="36sp" />
            <Button android:id="@+id/incTeamB" android:text="+1" />
        </LinearLayout>

    </LinearLayout>

    <Button
```

```
android:id="@+id/rstBtn"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:text="Reset" />
```

```
</LinearLayout>
```

6. What's Left in LinearLayout & Next Steps

You have now mastered ~90% of everything developers use in LinearLayout daily! The remaining ~10% consists of niche attributes (like `layout_weight` on custom views, or programmatic animation transitions), which are best learned when building complex custom views later.

Next Up: Moving to **FrameLayout** (Z-axis stacking, badges, overlays, and fragment containers).